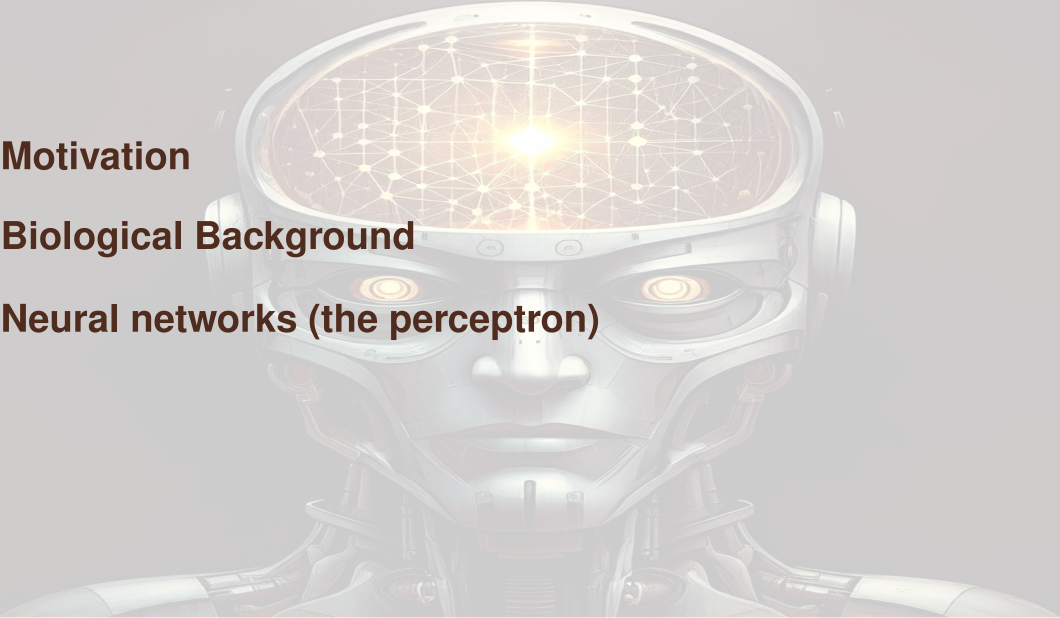
Clustering & Classification using ANNs

Adriana-Stefania Ciupeanu
University of Manitoba

The University of Manitoba campuses are located on original lands of Anishinaabeg, Ininew, Anisininew, Dakota and Dene peoples, and on the National Homeland of the Red River Métis. We respect the Treaties that were made on these territories, we acknowledge the harms and mistakes of the past, and we dedicate ourselves to move forward in partnership with Indigenous communities in a spirit of Reconciliation and collaboration.

Outline



- Motivation
- Biological Background
- Neural networks (the perceptron)

Motivation

Biological Background

Neural networks (the perceptron)

Why Neural Networks?

Applications:

- Image recognition
- Natural language processing
- Time series prediction
- Complex classification

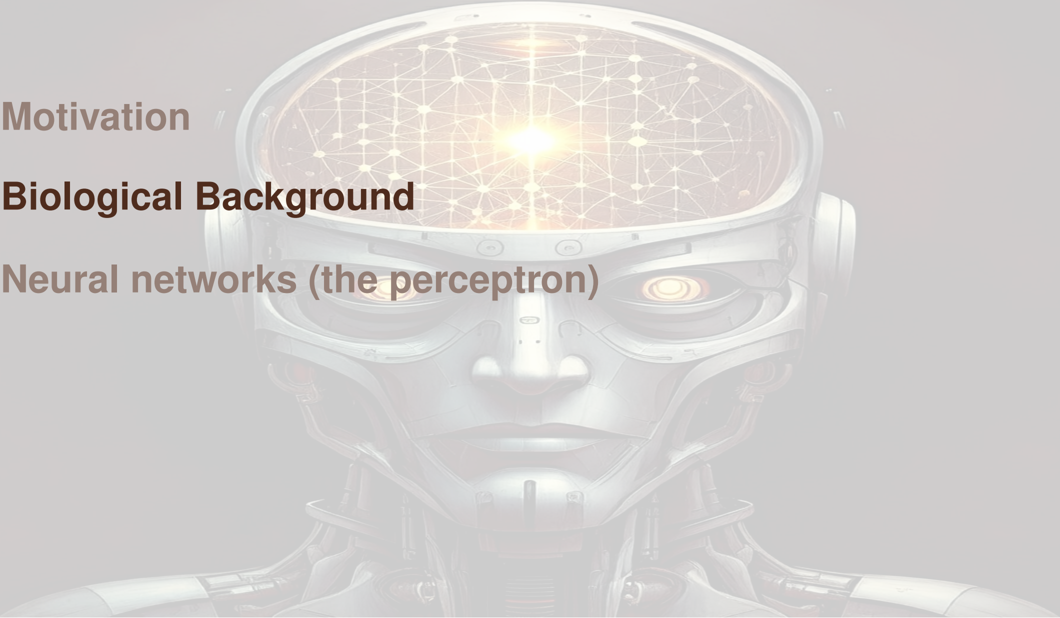
Key Advantages:

- Learn non-linear patterns
- Automatic feature learning
- Surprisingly flexible
- Proven in practice

Motivation

Biological Background

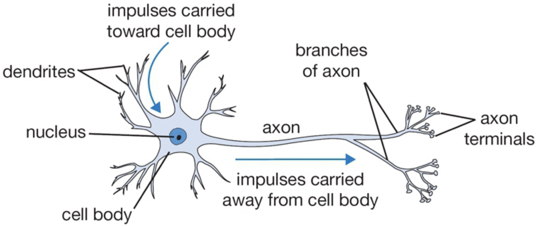
Neural networks (the perceptron)



The Biological Neuron

Many machine learning methods inspired by biology, e.g., the (human) brain

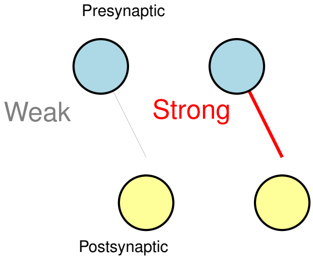
Our brain has 10^{11} neurons, each of which communicates (is connected) to 10^{4} other neurons



- **Dendrites**: receive signals
- **Cell body**: integrates signals
- **Axon**: transmits output
- **Synapses**: modifiable connections

Learning in Brains and Neural Networks: Synaptic Plasticity

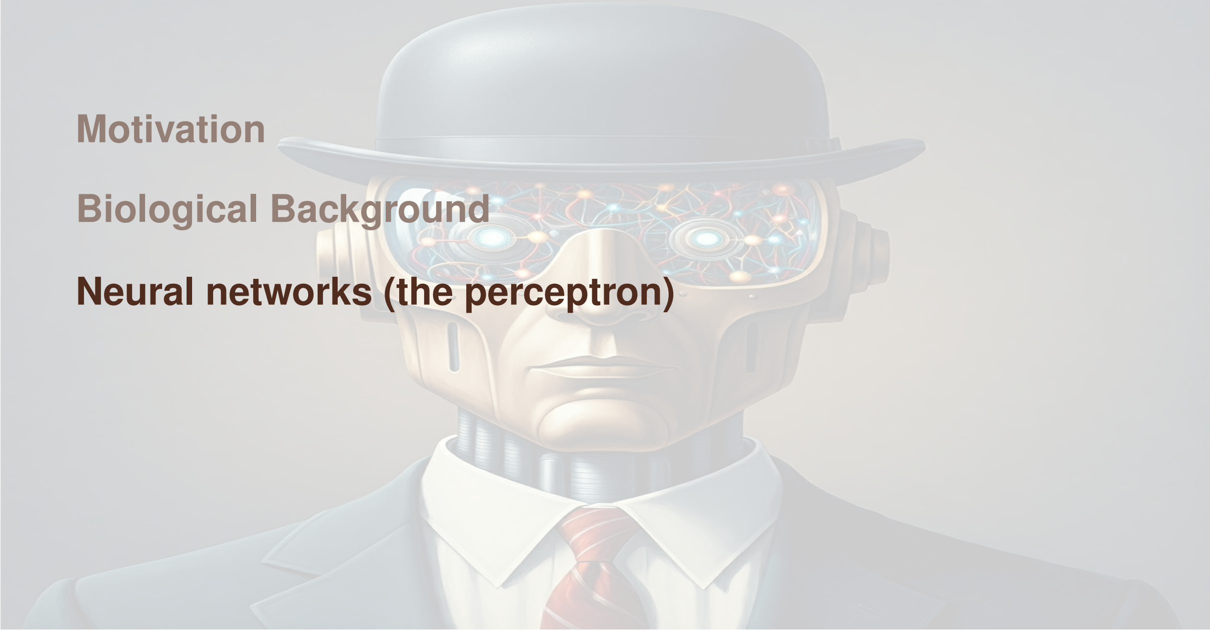
Synaptic Plasticity: The ability of connections between neurons to change strength based on activity.



Hebbian Principle: “Neurons that fire together, wire together”

Why this matters for artificial neural networks:

- Connection strength $\leftrightarrow$ **weights** in our network
- Learning means **adjusting weights** based on activity patterns
- Backpropagation is the artificial version of synaptic plasticity



Motivation

Biological Background

Neural networks (the perceptron)

Artificial neural network (ANN) - from Wikipedia

Artificial neural networks *(ANNs) are computing systems inspired by the biological neural networks that constitute animal brains*
An ANN is based on a collection of connected units or nodes called ***artificial neurons****, which loosely model the neurons in a biological brain. Each connection, like the synapses in a biological brain, can transmit a signal to other neurons. An artificial neuron receives signals then processes them and can signal neurons connected to it. The “signal" at a connection is a real number, and the output of each neuron is computed by some non-linear function of the sum of its inputs. The connections are called edges. Neurons and edges typically have a weight that adjusts as learning proceeds. The weight increases or decreases the strength of the signal at a connection. Neurons may have a threshold such that a signal is sent only if the aggregate signal crosses that threshold.*

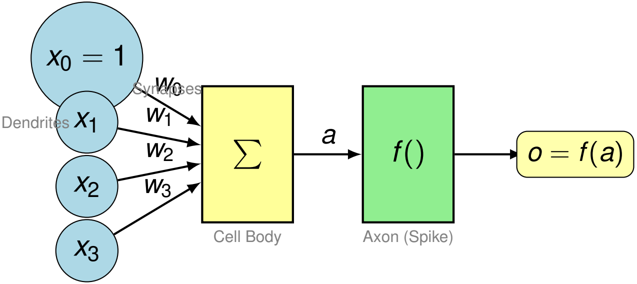
The perceptron

One of the first neural networks (invented 1943, implemented 1957), made for simple classification tasks, for example recognising letters or numbers

Two layers: the *input* layer (the *retina*) and the *output* layer

Inputs are 0 or 1, so are outputs

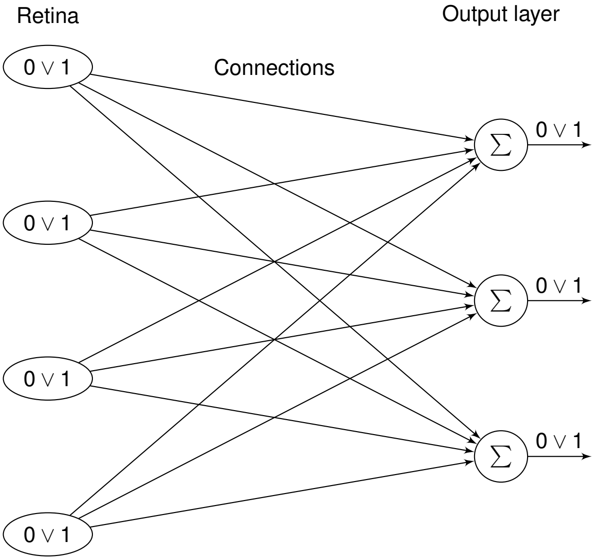
From Biology to Computation



$$a = \sum_{i=0}^{n} w_i x_i \quad \text{(weighted sum)}$$

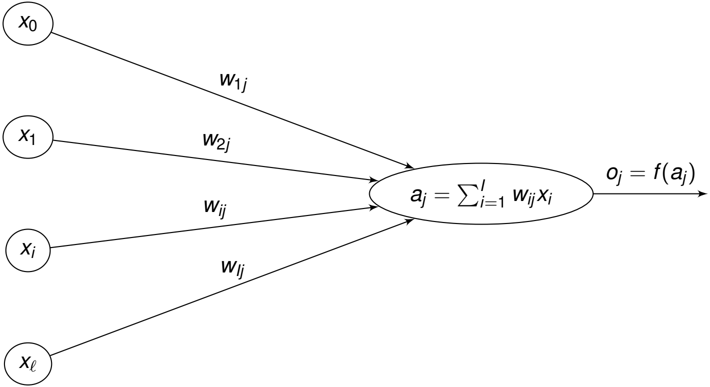
$$o = f(a) \quad \text{(activation)}$$

Biological Neuron	$\leftrightarrow$	**Artificial Neuron**
Dendrites receive signals	$\leftrightarrow$	Inputs x_i
Synaptic strength	$\leftrightarrow$	Weights w_i
Cell body integrates	$\leftrightarrow$	Summation $\sum w_i x_i$
Action potential threshold	$\leftrightarrow$	Activation $f(a)$
Axon fires/doesn't fire	$\leftrightarrow$	Output o



The connections into the output layer are called *synapses*, they are modifiable

The activation function



Here,

$$f(a_j) = \begin{cases} 0 & \text{if } a_j \leq 0 \\ 1 & \text{if } a_j > 0 \end{cases}$$

The activation function

We have I input neurons taking values 0 or 1, O output neurons taking values 0 or 1, weights $W = [w_{ij}] \in \mathcal{M}_{IO}$ and a threshold function f

More generally, use a threshold θ_j for each output neuron

$$o_j = \begin{cases} 0 & \text{if } a_j \leq \theta_j \\ 1 & \text{if } a_j > \theta_j \end{cases}$$

The thresholds (or *response bias*) and the weights are modifiable by learning. To do that easily for the threshold, consider an input neuron that is always on, say neuron 0, and set weights $w_{0j} = -\theta_j$, making the weights matrix an $(I + 1) \times O$-matrix

Another way to write the activation is

$$o_j = \begin{cases} 0 & \text{if } a_j + w_{0j} \leq 0 \\ 1 & \text{if } a_j + w_{0j} > 0 \end{cases}$$

where $w_{0j} = -\theta_j$

Common Activation Functions

Function	Definition	Range
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$(0, 1)$
Tanh	$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$	$(-1, 1)$
ReLU	$f(a) = \max(0, a)$	$[0, \infty)$
Linear	$f(a) = a$	$(-\infty, \infty)$

Key insight:

- Hidden layers: Use ReLU, tanh, or sigmoid
- Output layer: Match to task (linear, sigmoid, softmax)

Why Non-Linearity Matters

Without activation:

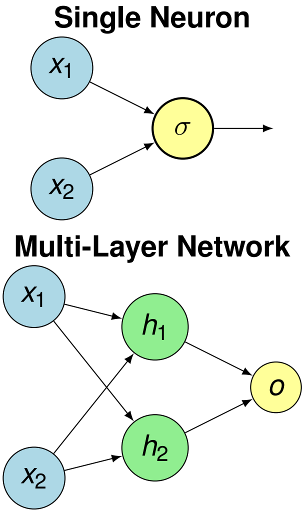
$$o = W_2(W_1 x) = (W_2 W_1)x = W_{\text{combined}} x \quad \text{(linear!)}$$

Result: Can only learn linear patterns

Solution: Apply non-linear function $f()$ after each sum

Single Neuron vs. Network

Limitation: Single neuron = linear decision boundary



The Learning Problem

Goal: Adjust weights to minimize prediction error

Forward Pass (predict)

$$\text{input} \xrightarrow{\text{weights}} \text{hidden} \xrightarrow{\text{weights}} \text{output}$$

Backward Pass (learn)

$$\text{Error} \xrightarrow{\text{gradients}} \text{update weights}$$

Loss function:

$$\text{Loss} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Learning something simple

The aim is to adjust the synaptic weights so that the proper response is provided to a given stimulus

Let us first do a simple example: the OR truth table

$$\begin{array}{cccc} 0 & 0 & \mapsto & 0 \\ 1 & 0 & \mapsto & 1 \\ 0 & 1 & \mapsto & 1 \\ 1 & 1 & \mapsto & 1 \end{array}$$

So we have two neurons in the retina and a single output neuron

Supervised learning

(From R. Rojas)

> ***Supervised learning****: method in which some input vectors are collected and presented to the network. The output computed by the network is observed and the deviation from the expected answer is measured*
>
> *The weights are corrected according to the magnitude of the error in the way defined by the learning algorithm*
>
> *Also called learning with a teacher, since a control process knows the correct answer for the set of selected input vectors*

Further distinctions in supervised learning methods

Methods with *reinforcement* or *error correction*

- **Reinforcement learning**: used when after each presentation of an input-output example, we only know whether the network produces the desired result or not. Weights are updated based on this information (i.e., the Boolean values true or false), so only the input vector can be used for weight correction
- In learning with **error correction**, the magnitude of the error, together with the input vector, determines the magnitude of the corrections to the weights. In many cases, we try to eliminate the error in a single correction step

A first learning algorithm

Suppose the training set consists of two sets of points P and N

- **start:** Generate random weight vector w_0; set $t := 0$
- **test:** A vector $x \in P \cup N$ is selected randomly
  - if $x \in P$ and $\langle w_t, x \rangle > 0$ go to **test**
  - if $x \in P$ and $\langle w_t, x \rangle \leq 0$ go to **add**
  - if $x \in N$ and $\langle w_t, x \rangle < 0$ go to **test**
  - if $x \in N$ and $\langle w_t, x \rangle \geq 0$ go to **subtract**
- **add:** set $w_{t+1} = w_t + x$ and $t := t + 1$, goto **test**
- **subtract:** set $w_{t+1} = w_t - x$ and $t := t + 1$, goto **test**

Widrow-Hoff learning rule

Need to provide the correct answer, i.e., this is a supervised learning rule

An output cell only learns if it is mistaken

Present random inputs and apply the rule if the output does not match the known output

Widrow-Hoff learning rule

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} + \eta(t_j - o_j)x_j = w_{ij}^{(t)} + \Delta w_{ij} \tag{1}$$

with

- Δw_{ij} correction to add to the weight w_{ij}
- x_i: value (0 or 1) of the ith retinal cell
- o_j: response of the jth output cell
- t_j target response (correct desired response)
- $w_{ij}^{(t)}$: weight of the synapse between the ith retinal cell and jth output cell at time t. Typically initiated at small random values
- η: small positive constant, the *learning constant*

Understanding Loss Functions

For a single data point with target y and network output o, we measure error as:

Squared error loss:

$$L = \frac{1}{2}(y - o)^2$$

For all N training samples:

$$L_{\text{total}} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{2}(y_i - o_i)^2$$

Goal of learning: Minimize this loss by adjusting weights

How do we minimize? We compute partial derivatives of the loss with respect to each weight, then move weights in the direction that decreases loss

Derivatives of Common Activation Functions

We'll need these when using the chain rule. For each activation $f(a)$, we need $\frac{\partial f}{\partial a}$:

Function $f(a)$	**Derivative** $f'(a)$	**In terms of output** $o = f(a)$
Sigmoid	$\sigma(a) = \frac{1}{1+e^{-a}}$	$\sigma'(a) = \sigma(a)(1 - \sigma(a))$ $= o(1 - o)$
Tanh	$\tanh(a)$	$\tanh'(a) = 1 - \tanh^2(a)$ $= 1 - o^2$
ReLU	$f(a) = \max(0, a)$	$f'(a) = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{if } a < 0 \end{cases}$

Key: We'll use these derivatives in the chain rule for backpropagation

Backpropagation: Single Layer Network

Simple case: One input layer, one output neuron (no hidden layer)

Forward pass:

$$a = w_1 x_1 + w_2 x_2 + w_0 \quad \text{(weighted sum)}$$
$$o = f(a) \quad \text{(apply activation, e.g., sigmoid)}$$

Loss:

$$L = \frac{1}{2}(y - o)^2$$

Goal: Find $\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial w_0}$ to update weights

Applying the Chain Rule

To find $\frac{\partial L}{\partial w_1}$, we use the chain rule:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial a} \cdot \frac{\partial a}{\partial w_1}$$

Break it into pieces:

- $\frac{\partial L}{\partial o} = \frac{\partial}{\partial o}\left[\frac{1}{2}(y-o)^2\right] = -(y-o)$
- $\frac{\partial o}{\partial a} = f'(a)$ (depends on activation function, e.g., $o(1-o)$ for sigmoid)
- $\frac{\partial a}{\partial w_1} = \frac{\partial}{\partial w_1}(w_1 x_1 + w_2 x_2 + w_0) = x_1$

The Gradient

Putting it together:

$$\frac{\partial L}{\partial w_1} = \underbrace{-(y-o)}_{\text{error}} \cdot \underbrace{f'(a)}_{\text{slope of activation}} \cdot \underbrace{x_1}_{\text{input}}$$

More generally, for any weight w_i:

$$\frac{\partial L}{\partial w_i} = -(y-o) \cdot f'(a) \cdot x_i$$

Similarly, for the bias:

$$\frac{\partial L}{\partial w_0} = -(y-o) \cdot f'(a) \cdot x_0 = -(y-o) \cdot f'(a)$$

(since $x_0 = 1$ is the bias input)

Gradient Descent Update

Once we have the gradient, we update weights using gradient descent:

$$w_i^{(t+1)} = w_i^{(t)} - \eta \frac{\partial L}{\partial w_i}$$

where η is the learning rate (small positive constant, like $\eta = 0.01$)

Substituting our gradient:

$$w_i^{(t+1)} = w_i^{(t)} - \eta \left[-(y - o) \cdot f'(a) \cdot x_i \right]$$

$$w_i^{(t+1)} = w_i^{(t)} + \eta (y - o) \cdot f'(a) \cdot x_i$$

Compare to Widrow-Hoff:

$$w_i^{(t+1)} = w_i^{(t)} + \eta (t - o) x_i$$

The extra $f'(a)$ term is what makes backprop work with smooth activations!

Comparison: Widrow-Hoff vs Backpropagation

Aspect	**Widrow-Hoff**	**Backpropagation**
Layers	Single layer only	**Any number of layers**
Update rule	$w \leftarrow w + \eta(t - o)x$	$w \leftarrow w - \eta \frac{\partial L}{\partial w}$
Activation	Step function (0/1)	**Smooth** (sigmoid, tanh, ReLU)
Chain rule	Not needed	**Essential**
Non-linearity	Ignored	**Accounted for via** $f'(a)$

Backpropagation is Widrow-Hoff generalized:

- Works with smooth activation functions (not just 0/1)
- Uses chain rule to compute gradients through multiple layers
- Error term $(t - o)$ becomes $-(y - o) \cdot f'(a)$ to account for saturation

Learning OR

Input	Input	$\to$	Target
0	0	$\mapsto$	0
1	0	$\mapsto$	1
0	1	$\mapsto$	1
1	1	$\mapsto$	1

Three cells in the retina (two inputs and the “dummy” cell used for the threshold) and one output cell. So inputs and outputs must be

Input	Input	Const	$\to$	Target
1	0	0	$\mapsto$	0
1	1	0	$\mapsto$	1
1	0	1	$\mapsto$	1
1	1	1	$\mapsto$	1

Initialise the 3×1 weight matrix W to zero:

$$W = (w_0 = -\theta, w_1, w_2)^T = (0, 0, 0)^T$$

Iteration 1: Present $[1, 1, 1] \to 1$

Predict:

$$a = w_0 \cdot 1 + w_1 \cdot 1 + w_2 \cdot 1 = 0 + 0 + 0 = 0$$

Since $a = 0 \leq 0$, output $o = 0$. But target is $t = 1$, so error is $1 - 0 = 1$.

Update weights:

$$\Delta w_0 = 0.1 \times 1 \times 1 = 0.1$$
$$\Delta w_1 = 0.1 \times 1 \times 1 = 0.1$$
$$\Delta w_2 = 0.1 \times 1 \times 1 = 0.1$$

New weights: $w = [0.1, 0.1, 0.1]^T$

Iteration 2: Present $[1, 0, 0] \to 0$

Predict:

$$a = 0.1 \cdot 1 + 0.1 \cdot 0 + 0.1 \cdot 0 = 0.1$$

Since $a = 0.1 > 0$, output $o = 1$. But target is $t = 0$, so error is $0 - 1 = -1$.

Update:

$$\Delta w_0 = 0.1 \times (-1) \times 1 = -0.1$$
$$\Delta w_1 = 0.1 \times (-1) \times 0 = 0$$
$$\Delta w_2 = 0.1 \times (-1) \times 0 = 0$$

New weights: $w = [0, 0.1, 0.1]^T$

And we are done!

With the weights

$$W = \begin{pmatrix} 0 \\ 0.1 \\ 0.1 \end{pmatrix}$$

we are done. Indeed

Input 0	Input 1	Input 2	a	o	Should be
1	0	0	0	0	0
1	1	0	0+0.1+0	1	1
1	0	1	0+0+0.1	1	1
1	1	1	0+0.1+0.1	1	1

Setting Up R

```r
# Install once
install.packages("neuralnet")

# Load for session
library(neuralnet)
```

The `neuralnet` package provides:

- Flexible network specification
- Training via backpropagation
- Prediction on new data
- Network visualization

Using `neuralnet` to learn OR

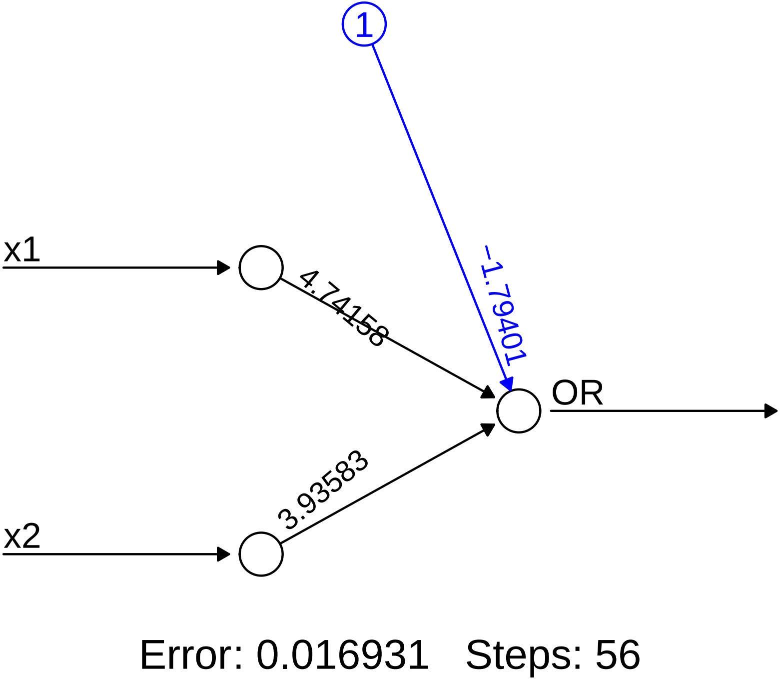
First, create the truth table

```
OR_table = matrix(c(0, 0, 0,
                    1, 0, 1,
                    0, 1, 1,
                    1, 1, 1),
                  nc = 3, byrow = TRUE)
OR_table = as.data.frame(OR_table)
colnames(OR_table) = c("x1", "x2", "OR")
```

Now create and train the NN

The “formula” is to find the OR column using the x1 and x2 columns. We use no hidden layer

```
nn_OR = neuralnet(OR ~ x1 + x2,
                  data = OR_table,
                  act.fct = "logistic",
                  hidden = 0,
                  linear.output = FALSE)
# Plot the result
plot(nn_OR, rep = "best")
```


1
x1
x2
4.74158
3.93583
-1.79401
OR
Error: 0.016931   Steps: 56

Testing the result

```
pred = predict(nn_OR, OR_table)
OR_table$result = pred > 0.5
kable(OR_table, "latex", booktabs = TRUE)
```

x1	x2	OR	result
0	0	0	FALSE
1	0	1	TRUE
0	1	1	TRUE
1	1	1	TRUE

Learning XOR

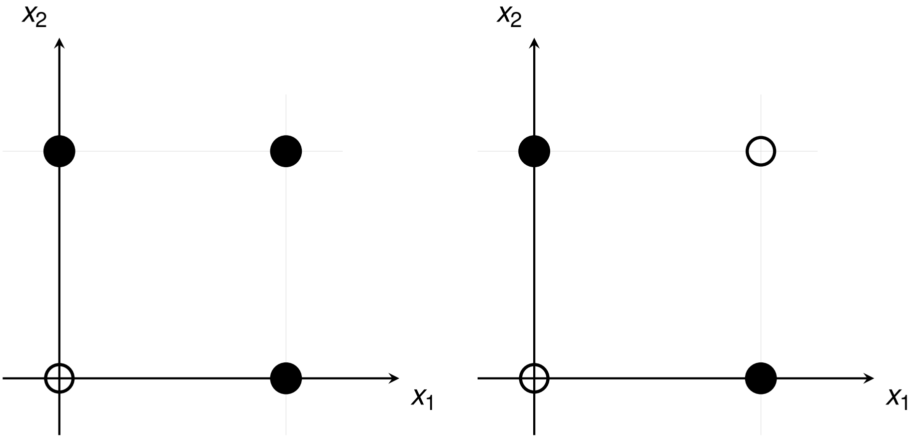
Let us now look at the XOR truth table

$$\begin{array}{ccccc} 0 & 0 & \mapsto & 0 \\ 1 & 0 & \mapsto & 1 \\ 0 & 1 & \mapsto & 1 \\ 1 & 1 & \mapsto & 0 \end{array}$$

This problem is not solvable with a simple perceptron of the type we just used, as truth table is not *linearly separable*

Indeed, we would get weights $w_1 > 0$, $w_2 > 0$ to activate when presenting $[1, 0]$ and $[0, 1]$, but would require that the sum of the weights when applied to the input $[1, 1]$, give a negative value.

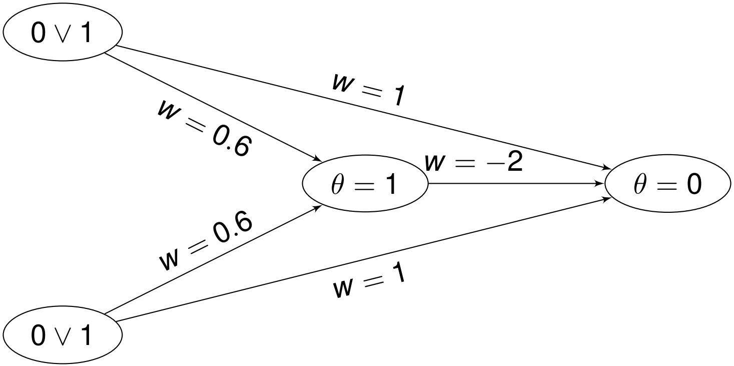
Linear separability and OR and XOR



A single-layer perceptron can only learn linearly separable problems

Adding a hidden layer

It is possible to do XOR, but we need to add a **hidden layer**



Now the XOR truth table

```
XOR_table = matrix(c(0, 0, 0,
                     1, 0, 1,
                     0, 1, 1,
                     1, 1, 0),
                  nc = 3, byrow = TRUE)
XOR_table = as.data.frame(XOR_table)
colnames(XOR_table) = c("x1", "x2", "XOR")
```

Try to learn it without a hidden layer

```r
nn_XOR = neuralnet(XOR ~ x1 + x2,
                   data = XOR_table,
                   act.fct = "logistic",
                   hidden = 0,
                   linear.output = FALSE)
pred = predict(nn_XOR, XOR_table)
XOR_table$result = pred > 0.5
kable(XOR_table, "latex", booktabs = TRUE)
```

x1	x2	XOR	result
0	0	0	FALSE
1	0	1	TRUE
0	1	1	FALSE
1	1	0	TRUE

Now with a hidden layer

```r
nn_XOR = neuralnet(XOR ~ x1 + x2,
                   data = XOR_table,
                   act.fct = "tanh",
                   hidden = 1)
pred = predict(nn_XOR, XOR_table)
XOR_table$result = pred > 0.5
kable(XOR_table, "latex", booktabs = TRUE)
```

x1	x2	XOR	result
0	0	0	FALSE
1	0	1	TRUE
0	1	1	TRUE
1	1	0	TRUE

Should look into options.. :)

An example from the `neuralnet` manual – Training vs testing sets

`iris` is a built-in `R` dataset detailing physical characteristics of 150 flowers from 3 iris species

```
train_idx <- sample(nrow(iris), 2/3 * nrow(iris))
iris_train <- iris[train_idx, ]
iris_test <- iris[-train_idx, ]
```

Thus we pick at random 2/3 of the data for training and 1/3 for testing. See some considerations on training, validation and testing on this Wikipedia page

An example from the `neuralnet` manual

```
nn <- neuralnet(Species == "setosa" ~ Petal.Length + Petal.Width,
                iris_train, linear.output = FALSE)
pred <- predict(nn, iris_test)
table(iris_test$Species == "setosa", pred[, 1] > 0.5)

##
##          FALSE TRUE
##   FALSE     37    0
##   TRUE       0   13
```

Another example – multiclass classification

```
nn <- neuralnet((Species == "setosa") +
                  (Species == "versicolor") +
                  (Species == "virginica")
                 ~ Petal.Length + Petal.Width,
                iris_train, linear.output = FALSE)
pred <- predict(nn, iris_test)
table(iris_test$Species, apply(pred, 1, which.max))
```

```
##
##                1
##   setosa      13
##   versicolor 19
##   virginica  18
```